

15.1 Binary Search Trees

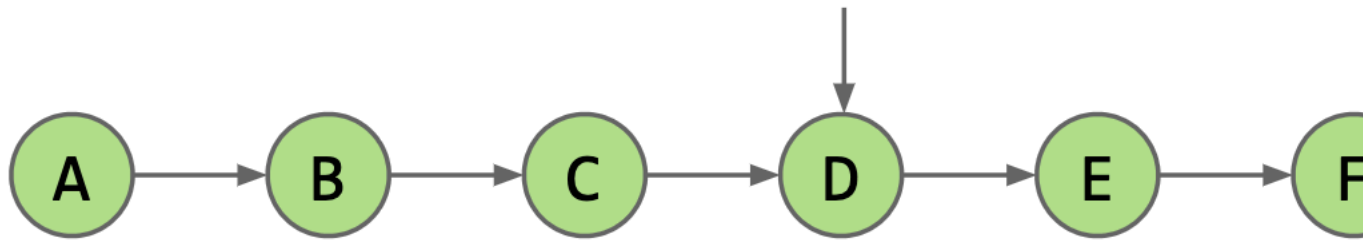
Linked Lists are great, but it takes a long time to search for an item, even if the list is sorted! What if the item is at the end of the list? That would take linear time!

We know that for an array, we can use binary search to find an element faster. Specifically, in $\log n$ time. For a short explanation of binary search, check out this [link](#).

In binary search, we know the list is sorted, so we can use this information to narrow our search. First, we look at the middle element. If it is bigger than the element we are searching for, we look to the left of it. If it is smaller than the element we are searching for, we look to the right. We then look at the middle element of the respective halves and repeat the process until we find the element we are looking for (or don't find it because the list doesn't contain it).

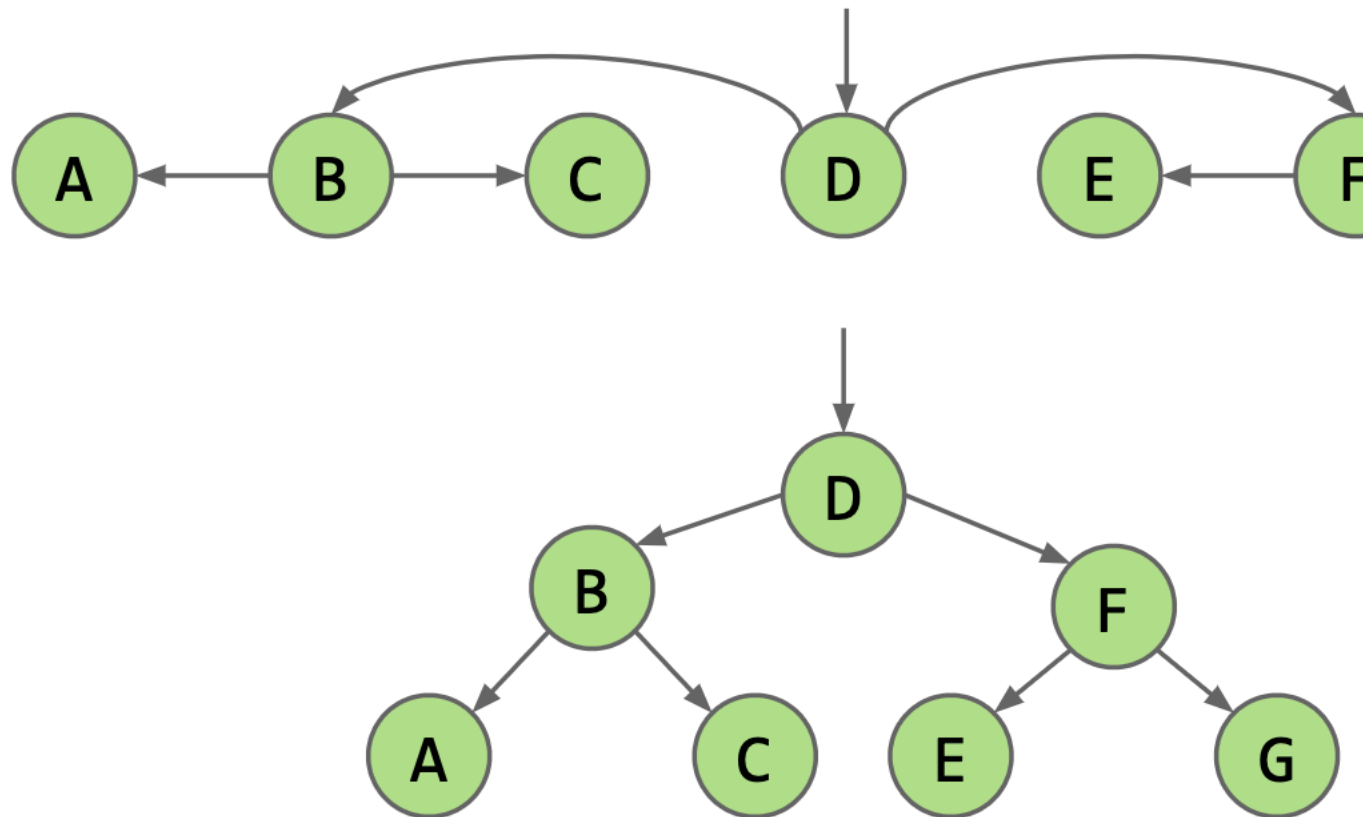
But how do we run binary search for a linked list? We would have to traverse all the way to the middle in order to check the element there, which takes linear time just on its own!

One optimization we can implement is to have a reference to the middle node. This way, we can get to the middle in constant time. Then, if we flip the nodes' pointers, which allows us to traverse to both the left and right halves, we can decrease our runtime by half!



A linked list with a middle pointer

But we can further optimize by adding pointers to the middle of each recursive half like so. Now, if you stretch this structure vertically, you will see a tree! This specific tree is called a **binary tree** because each juncture splits in 2.



A linked list with recursive middle pointers is a binary tree!

{% embed url="https://www.youtube.com/watch?v=AcRKQOe0zYg" %}

15.2 BST Definitions

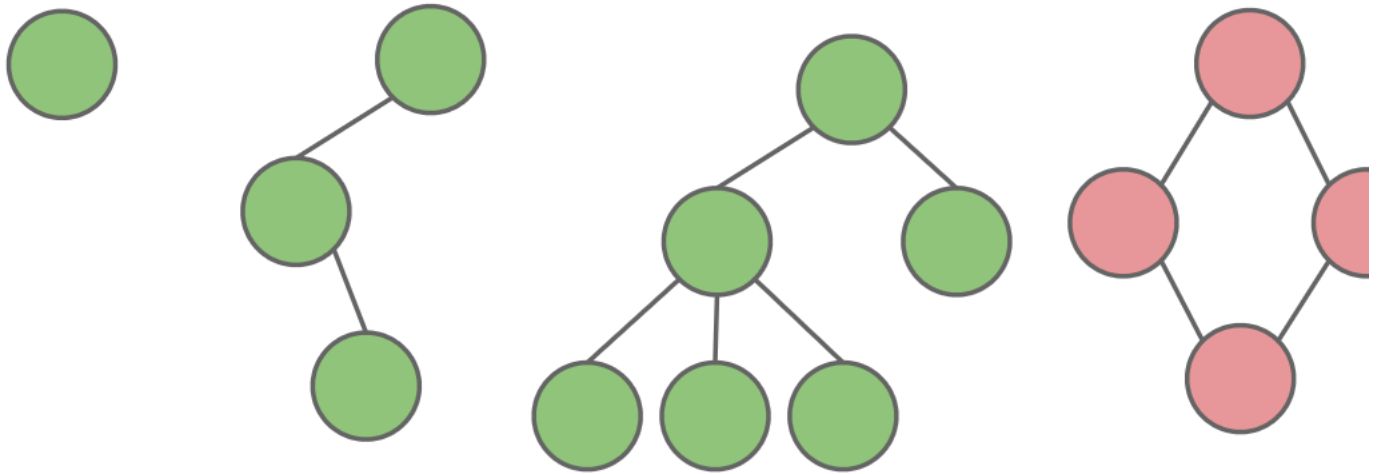
Trees are composed of nodes and edges that connect those nodes.

Constraint: there is only one path between any two nodes.

In some trees, we select a **root** node which is a node that has no parents.

A tree also has **leaves**, which are nodes with no children.

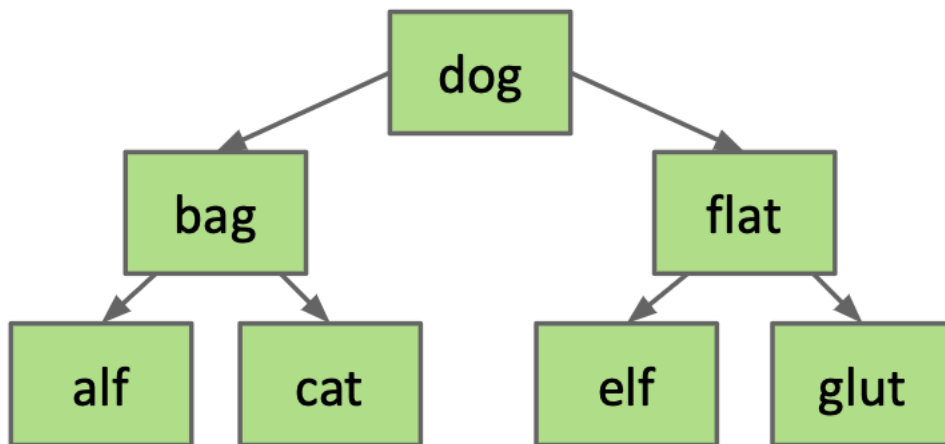
In the picture below, the green structures are valid trees, while the pink structure is not (since it has a cycle).



Examples of valid and invalid trees

Relating this to the original tree structure we came up with earlier, we can now introduce new constraints to the already existing constraints. This creates more specific types of trees, two examples being Binary Trees and Binary Search Trees.

- **Binary Trees:** in addition to the above requirements, also hold the binary property constraint. That is, each node has either 0, 1, or 2 children.
- **Binary Search Trees:** in addition to all of the above requirements, also hold the property that For every node X in the tree:
 - Every key in the left subtree is less than X's key.
 - Every key in the right subtree is greater than X's key. ****Remember this property!!** We will reference it a lot throughout the duration of this module and 61B.



A valid BST: every key in the left subtree is smaller than its parent, and every key in the right subtree is larger.

Here is the BST module we'll be using for the rest of this chapter:

```
private class BST<Key> {
    private Key key;
    private BST left;
    private BST right;

    public BST(Key key, BST left, BST Right) {
        this.key = key;
        this.left = left;
        this.right = right;
    }

    public BST(Key key) {
        this.key = key;
    }
}
```

15.3 BST Operations

Search

To search for something, we employ binary search, which is made easy due to the BST property.

We know that the BST is structured such that all elements to the right of a node are greater and all elements to the left are smaller. Knowing this, we can start at the root node and compare it with the element, X , that we are looking for. If X is greater to the root, we move on to the root's right child. If its smaller, we move on to the root's left child. We repeat this process recursively until we either find the item or we get to a leaf, in which case the tree does not contain the item.

```
static BST find(BST T, Key sk) {
    if (T == null)
        return null;
    if (sk.equals(T.key))
        return T;
    else if (sk < T.key)
        return find(T.left, sk);
    else
        return find(T.right, sk);
}
```

If our tree is relatively “bushy”, the find operation will run in $\log n$ time because the height of the tree is $\log n$.

Insert

We **always** insert at a leaf node!

First, we search in the tree for the node. If we find it, then we don't do anything. If we don't find it, we will be at a leaf node already. At this point, we can just add the new element to either the left or right of the leaf, preserving the BST property.

{% embed url="https://www.youtube.com/watch?v=otDvoMb8UqE" %}

```
static BST insert(BST T, Key ik) {
    if (T == null)
        return new BST(ik);
    if (ik < T.key)
        T.left = insert(T.left, ik);
    else if (ik > T.key)
        T.right = insert(T.right, ik);
    return T;
}
```

Delete

Deleting from a binary tree is a little bit more complicated because whenever we delete, we need to make sure we reconstruct the tree and still maintain its BST property. Let's break this problem down into three categories:

- the node we are trying to delete has no children
- has 1 child
- has 2 children

Deletion: No Children

If the node has no children, it is a leaf, and we can just delete its parent pointer and the node will eventually be swept away by the [garbage collector](#).

Deletion: One Child

If the node only has one child, we know that the child maintains the BST property with the parent of the node because the property is recursive to the right and left subtrees. Therefore, we can just reassign the parent's child pointer to the node's child and the node will eventually be garbage collected.

Deletion: Two Children

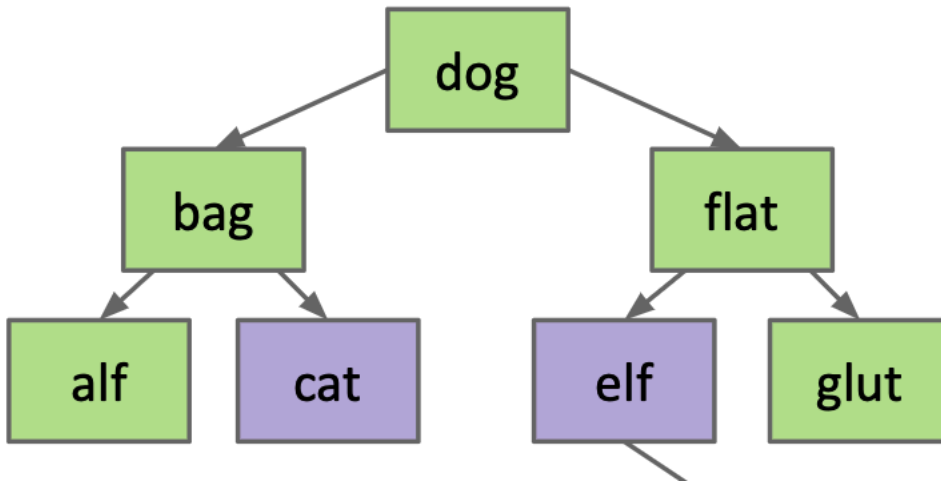
If the node has two children, the process becomes a little more complicated because we can't just assign one of the children to be the new root. This might break the BST property.

Instead, we choose a new node to replace the deleted one.

We know that the new node must:

- be $>$ than everything in left subtree.
- be $<$ than everything right subtree.

In the below tree, we show which nodes would satisfy these requirements given that we are trying to delete the *dog* node.



Possible candidates to replace *dog* after deletion

To find these nodes, you can just take the right-most node in the left subtree or the left-most node in the right subtree. Then, we replace the *dog* node with either *cat* or *elf* and then remove the old *cat* or *elf* node.

This is called **Hibbard deletion**, and it gloriously maintains the BST property amidst a deletion.

15.4 BSTs as Sets and Maps

We can use a BST to implement the Set ADT. If we use a BST, we can decrease the runtime of *contains* to $\log(n)$ because of the BST property which enables us to use binary search!

We can also make a binary tree into a map by having each BST node hold (key, value) pairs instead of singular values. We will compare each element's key in order to determine where to place it within our tree.

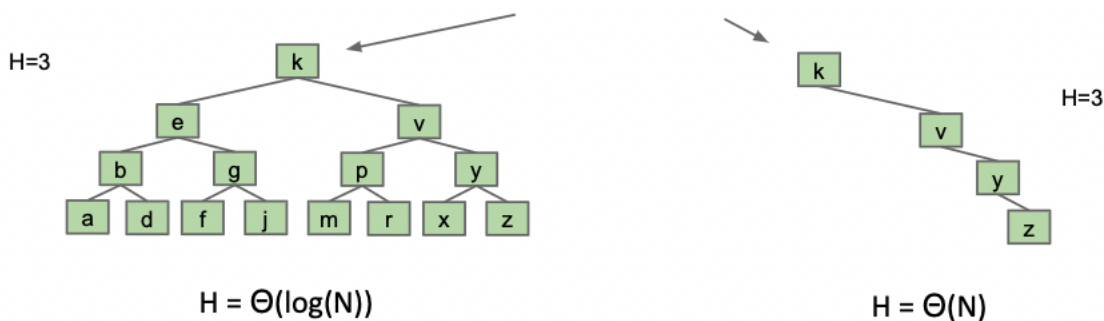
15.5 BST Performance

Tree Height

One unfortunate feature of BSTs is that they range from a best-case “bushy” tree to a worst-case “spindly” tree.

In the best case, our tree will have height $\Theta(\log N)$, whereas in the worst case our tree has a height of $\Theta(N)$, at which point it basically becomes a linked list. For example, *contains* on a “spindly” BST would take linear time.

Both trees below have a height $H = 3$, yet the left tree is able to hold many more items than the left.



15.6 Big O vs. Worst Case

Consider the following statements about BSTs. Which of the following are true?

1. The worst-case height of a BST is $\Theta(N)$.
2. BST height is $O(N)$.
3. BST height is $O(N^2)$.

The answer is that all three statements are true. BSTs always have a height that is linear or better, and a linear height is obviously “less than” the quadratic upper bound in the last point.

However, a more tricky question is which of the three statements is *the most informative*.

The answer here is the first statement. Theta is always a more informative statement than big O.

Think about theta as similar to $\approx$. When we say some function $F(N) \in \theta(N)$ we mean it grows exactly linearly.

By contrast, you should think of O as $\leq$. If we say some function $f(N) \in O(N)$, then we mean it grows at a linear rate or slower.

As a specific example, suppose we know that $f(N)$ is one of these functions: $3N$, $4\log N + 6$. If I tell you that $f(N) \in O(N)$ you have no idea which function we're talking about. But if I say $f(N) \in \theta(N)$, you know immediately which one we mean.

For an analogy, consider the following statements about the worst-case cost of a hotel room:

1. The most expensive room is \$639/night.
2. The most expensive room is less than or equal to \$2000/night.

Here, we see that the first statement gives us exact information, whereas the second statement does not. In the second statement, the most expensive room could be \$2000, \$10, or anywhere in between.

However, *both are statements about the worst case*. Applying this to asymptotic notation, this means that we can refer to the worst case with Θ , O , or even Ω . **Big O is not the same as the worst case!**

Using Big O

If Θ is always more informative than O , then why do we bother using Big O notation at all? There are several reasons:

- We can make broader statements. For example, saying “binary search is $O(\log N)$ ” is correct, but saying “binary search tree is $\Theta(\log N)$ ” would not be correct, since it can be constant in certain scenarios.
- Sometimes, it is not possible or extremely difficult to determine the exact runtime. In such cases, we would still like to provide a generalized upper bound.

15.7 Summary

Abstract data types (ADTs) are defined in terms of operations, not implementation. Several useful ADTs:

- Disjoint Sets, Map, Set, List.
- Java provides Map, Set, List interfaces, along with several implementations.

We've seen two ways to implement a Set (or Map):

- ArraySet: $\Theta(N)$ operations in the worst case.
- BST: $\Theta(\log N)$ operations if tree is balanced.

BST Implementations:

- Search and insert are straightforward (but insert is a little tricky).
- Deletion is more challenging. Typical approach is “Hibbard deletion”.

15.8 Exercises

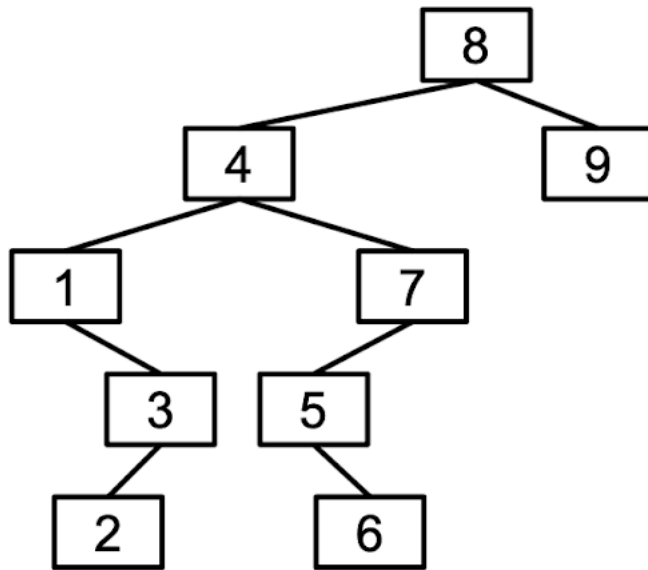
Factual

1. What is the best and worst-case height of a BST?

► Problem 1

Procedural

1. Suppose that a certain BST has keys that are integers between 1 and 10. During the search for 5, which of the following sequences of keys are possible?
 - ☐ 10, 9, 8, 7, 6, 5
 - ☐ 4, 10, 8, 7, 5, 3
 - ☐ 1, 10, 2, 9, 3, 8, 4, 7, 6, 5
 - ☐ 1, 2, 6, 8, 9, 5
2. Consider the below BST. What is the result after deleting 4 using Hibbard deletion, choosing the successor as the replacement?

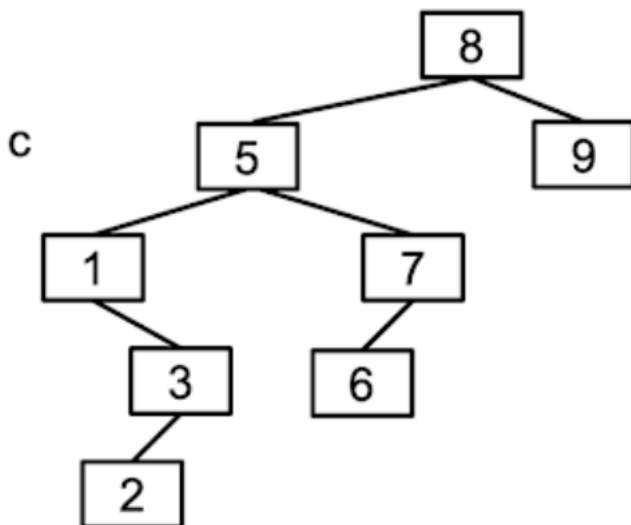


3. Suppose we implement the Stack ADT using an array. What is the worst case runtime of a push operation with this underlying data structure?

▼ Problem 1

- ☒ 10, 9, 8, 7, 6, 5: possible; this is the situation where we have a worst-case linear BST.
- ☐ 4, 10, 8, 7, 5, 3: not possible; we terminate our search once we reach the desired node.
- ☒ 1, 10, 2, 9, 3, 8, 4, 7, 6, 5: possible; the idea is that we should always search in the correct “direction” of our target node. If our target node is greater than our current node, then we should go to the right, and our next node should be larger. If our target node is less than our current node, then we should go to the left, and our next node should be smaller.
- ☐ 1, 2, 6, 8, 9, 5: not possible; note that this violates the constraint described above. When we reach 8, we should move to its left branch since our target node 5 is smaller, so we would never search 9.

▼ Problem 2



▼ Problem 3

The worst-case runtime is $\Theta(N)$, since a push might cause us to resize the underlying array.

Metacognitive

1. If inserting our data into a BST in random order yields $\log N$ height with high probability, why don't we just shuffle our data before inserting into the BST?
2. When we do Hibbard deletion from a BST, we always choose the successor as a replacement. The successor is guaranteed to only have zero or one child—why?

▼ Problem 1

Often in real-world applications, we don't have all our data at once. For example, imagine you're collecting time-based data that you insert into a BST each time a new value is reported. There is no easy way to shuffle your data when you only get one or a few points at a time.

▼ Problem 2

By definition, the successor is the maximum value in the subtree. Suppose, for the sake of contradiction, that the successor had two children. Then, it is not the maximum, since it is less than its right child. This is a contradiction, since we said the successor is the maximum value in the subtree. As such, the successor is guaranteed to have one child or less (if it has one child, it is its left child). Previous 16.6 Summary Next 17. Tree Traversals and Graphs

15. Binary Search Trees

In this Chapter we will discuss:

- Binary Search Tree
- BST Definitions
- BST Operations
- Sets vs. Maps, Summary